Análisis y diseño de un modelo de datos relacional

Introducción

Para el desarrollo de esta lectura, nos remitiremos a los elementos vistos en la lectura 3 del primer módulo, donde se presentaron los elementos de un modelo relacional, como las tablas, atributos, claves primaria y foránea, relaciones, cabecera y cuerpo de una tabla, grado, cardinalidad y dominio. Veremos más definiciones del proceso de análisis y diseño de los modelos de datos relacionales.

1. Análisis y diseño de un modelo de datos relacional

En la lectura mencionada tomamos, a través de la ingeniería reversa, un conjunto de tablas ya creadas y lo volcamos a un modelo de tablas, columnas y relaciones. En este punto, presentaremos un caso en donde tenemos que llegar a un modelo de tablas con sus tablas, columnas, claves, de las que ya conocemos sus características, pero que repasaremos a continuación.

Repaso de las definiciones que debemos aplicar

Hay tres nociones básicas que emplea el modelo de entidad – relación (DER):

- Entidades.
- Atributos.
- Relaciones.

Entidades

“Una entidad es [una clase o categoría de] «cosa» u «objeto» en el mundo real que es distinguible de todos los demás objetos” (Silberschatz, Korth y Sudarshan, 2002, p. 19). Por ejemplo, cada persona en un desarrollo es una instancia de la entidad persona. Esta instancia de entidad tiene un conjunto de atributos, y los valores para uno o varios atributos pueden servir para identificar una fila de forma unívoca (Silberschatz et al., 2002). “Por ejemplo, el DNI 67 789 901 identifica unívocamente una persona particular en la empresa” (Silberschatz et al., 2002, p. 19). Otro ejemplo: se puede pensar en los préstamos bancarios como un conjunto que define una entidad. Un atributo como el número de préstamo en la sucursal de Córdoba identifica unívocamente a un préstamo del

conjunto de préstamos que hace a la entidad *préstamos*. Una entidad puede ser concreta, como un conjunto de personas o de libros, o puede ser abstracta, como préstamos, viajes o un concepto como alquiler (Silberschatz et al., 2002).

Las instancias individuales de estas entidades comparten las mismas propiedades o atributos. “El conjunto de todas las personas que son clientes en un banco dado, por ejemplo, se pueden definir como el conjunto de entidades *cliente*. Análogamente, el conjunto de préstamos representa a los préstamos concedidos por un banco particular” (Silberschatz et al., p. 19).

> Por ejemplo, es posible definir el conjunto de entidades de todos los empleados de un banco (*empleado*) y el conjunto de todos los clientes del banco (*cliente*). Una entidad persona puede ser definida como parte de la entidad empleado y de otra entidad *cliente*, ambas cosas o ninguna. (Silberschatz et al., 2002, p. 19).

Una entidad se representa como una tabla y esta contiene un conjunto de atributos que podemos llamar columnas, asignados a filas para describir cada uno de sus integrantes en instancias o filas. Los atributos describen propiedades que posee cada integrante de la entidad. La creación de un atributo o columna para una tabla expresa que la base de datos almacena información similar concerniente a cada fila de la tabla, y cada fila puede tener su propio valor para cada columna (Silberschatz et al., 2002).

Algunas columnas de la tabla cliente:

> Son *id-cliente, nombre-cliente, calle-cliente y ciudad-cliente*. En la vida real, habría más atributos, tales como el número de la calle, el número del portal, la provincia, el código postal, pero no se incluyen en el ejemplo simple. Posibles atributos del conjunto de entidades préstamo son número-préstamo e importe. (Silberschatz et al., 2002, p. 19).

Cada fila tiene un valor para cada uno de sus atributos. Por ejemplo, una fila de cliente en concreto puede tener el valor 32 112 312 para id-cliente, el valor Santos para nombre-cliente, el valor Colón para calle-cliente y el valor Rosario para ciudad-cliente (Silberschatz et al., 2002).

La columna *id-cliente*:

> Se usa para identificar unívocamente a los clientes, dado que no hay más de un cliente con el mismo nombre, calle y ciudad. En los Estados Unidos, muchas empresas encuentran conveniente usar el número de seguridad social de una persona como un atributo [o columna] cuyo valor identifica unívocamente a la persona. En general, la empresa tendría que crear y asignar un identificador a cada cliente. (Silberschatz et al., 2002, pp. 19-20).

Es decir, aplicar el concepto de clave primaria visto con anterioridad en la materia.

Para cada columna o atributo hay un conjunto de valores permitidos, llamados el dominio (visto en el primer módulo de la materia) o el conjunto de valores de ese atributo (Silberschatz et al., 2002).

> El dominio del atributo *nombre-cliente* podría ser el conjunto de todas las cadenas de texto de una cierta longitud. Análogamente, el dominio del atributo *número-préstamo* podría ser el conjunto de todas las cadenas de la forma «P-n», donde n es un entero positivo. (Silberschatz et al., 2002, p. 20).

Una base de datos contiene una colección de conjuntos de filas para cada tabla, cada una de las cuales contiene un número de filas del mismo tipo. En la tabla 1, “se muestra parte de una base de datos de un banco que consta de dos conjuntos de entidades, cliente y préstamo” (Silberschatz et al., 2002, p. 20).

Tabla 1: Ejemplo de tabla *clientes*

Id cliente	Nombre - Cliente	Dirección	Ciudad	Teléfonos
32.112.312	Mario Santos	Colón 278	Rosario	4615666,4206633
46.566.778	Ana María Suarez Paz	La paz 676	Rosario	4813346
16.083.322	Roxana Becú Gómez	General Paz 44	Córdoba	4252500,4252501

Fuente: Adaptación propia de Silberschatz et al., 2002, p. 20.

Tabla 1. Vemos lo descripto anteriormente de columnas y filas de la tabla *clientes*.

Un atributo o columna se puede caracterizar por los siguientes tipos:

☰ Atributos simples y compuestos

En los ejemplos considerados hasta ahora, los atributos han sido simples, es decir, no están divididos en subpartes. Los atributos compuestos, en cambio, se pueden dividir en subpartes (es decir, en otros atributos). Por ejemplo, nombre-cliente podría estar estructurado como un atributo compuesto consistente en nombre, primer-apellido y segundo-apellido. Usar atributos compuestos en un esquema de diseño es una buena elección si el usuario desea referirse a un atributo completo en algunas ocasiones y, en otras, a algún componente del atributo. Se podrían haber sustituido los atributos del

conjunto de entidades *cliente*, *calle-cliente* y *ciudad-cliente*, por el atributo compuesto *dirección-cliente*, con los atributos *calle*, *ciudad*, *provincia* y *código-postal*. Los atributos compuestos ayudan a agrupar los atributos relacionados, esto hace a los modelos más claros.

Nótese también que un atributo compuesto puede aparecer como una jerarquía. Volviendo al ejemplo del atributo compuesto *dirección-cliente*, su componente *calle* puede ser, a su vez, dividido en *número-calle*, *nombre-calle* y *piso*. (Silberschatz et al., 2002, p. 20).

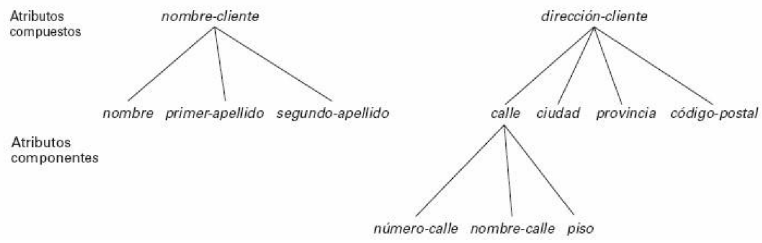
Estos ejemplos de atributos compuestos para el conjunto de entidades cliente se representan en la figura 1.

☰ Atributos monovalorados y multivalorados

Algunos atributos que se han especificado en los ejemplos tienen un valor único para una fila dada. Por ejemplo, el atributo *código-cliente*, para un cliente, referencia a un único número de cliente (Silberschatz et al., 2002).

> Tales atributos se llaman monovalorados. Puede haber ocasiones en las que un atributo tiene un conjunto de valores para una entidad específica. Considérese un conjunto de entidades empleado con el atributo *número-teléfono*. Cualquier empleado particular puede tener cero, uno o más números de teléfono. Este tipo de atributo se llama multivalorado. En ellos, se pueden colocar apropiadamente límites inferior y superior en el número de valores en el atributo multivalorado. (Silberschatz et al., 2002, p. 20).
>
> Como otro ejemplo, un atributo *nombre-subordinado* de la tabla *empleado* sería multivalorado, ya que un empleado en concreto podría tener cero, uno o más subordinados (Silberschatz et al., 2002).

Figura 1: Tipos de atributos compuestos



Fuente: Silberschatz et al., 2002, p. 21.

Figura 1. Tipos de atributos compuestos. Se muestra un ejemplo en el que los atributos inicialmente definidos pueden contener varios valores simples, que puede decidirse que se mantienen juntos o crear otros atributos simples para distribuirlos. Esto se decide en un proceso especial de análisis.

Cuando sea apropiado, se pueden establecer límites superior e inferior en el número de valores de un atributo multivalorado. Por ejemplo, un banco puede limitar el número de números de teléfono almacenados para un único cliente a dos. Colocando límites, en este caso, se expresa que el atributo *número-teléfono* de la tabla *clientes* puede tener entre cero y dos valores.

Atributos derivados. El valor para este tipo de atributo se puede derivar de los valores de otros atributos o filas relacionadas. Por ejemplo, sea el conjunto de entidades [la tabla] *cliente* que tiene un atributo *préstamos* que representa cuántos préstamos tiene un cliente en el banco. Ese atributo se puede derivar contando el número de entidades [filas] *préstamo* asociadas con ese cliente. Como otro ejemplo, considérese que el conjunto de entidades [la tabla clientes] tiene un atributo edad, que indica la edad del cliente. Si el conjunto de entidades [la tabla] *cliente* tiene también un atributo *fecha-de-nacimiento*, se puede calcular la edad a partir de *fecha-de-nacimiento* y de la fecha actual. Así, edad es un atributo derivado. En este caso, *fecha-de-comienzo* y *antigüedad* pueden serlo, ya que representan el primer día en el que el empleado comenzó a trabajar para el banco y el tiempo total que el empleado lleva trabajando para el banco, respectivamente. El valor de antigüedad se puede derivar del valor de *fecha-comienzo* y de la fecha actual. En este caso, *fecha-comienzo* se puede conocer como atributo base o atributo almacenado. El valor de un atributo derivado no se almacena, sino que se calcula cuando sea necesario.

Un atributo toma un valor nulo cuando una entidad [fila] no tiene un valor para un atributo. El valor nulo también puede indicar «no aplicable», es decir, que el valor no existe para la [fila] entidad. Por ejemplo, una persona puede no tener segundo nombre de pila. Nulo puede también designar que el valor de un atributo es desconocido. Un valor desconocido puede ser perdido (el valor existe, pero no se tiene esa información) o desconocido (no se conoce si el valor existe realmente o no). Por ejemplo, si el valor nombre para un cliente particular es nulo, se asume que el valor es perdido, ya que cada cliente debe tener un nombre. Un valor nulo para el atributo *piso* podría significar que la dirección no incluye un piso (no aplicable), que existe piso, pero no se conoce cuál es (perdido) o que no se sabe si el piso forma parte o no de la dirección del cliente (desconocido). (Silberschatz et al., 2002, p. 21).

Una base de datos para una empresa bancaria puede incluir diferentes tablas. Por ejemplo, además del mantenimiento de clientes y préstamos, el banco también

proporciona cuentas, que se representan mediante la tabla *cuenta* con atributos *número-cuenta* y *saldo* (Silberschatz et al., 2002). “También, si el banco tiene un número de sucursales diferentes, se puede mantener información acerca de todas las sucursales del banco” (Silberschatz et al., 2002, p. 21). La tabla *sucursal* se describe mediante los atributos *nombre-sucursal*, *ciudad-sucursal y activo.*

Tabla 2: Tabla de equivalencias de términos

Relación Momentos o etapas en el diseño de una base de datos relacional		
Diseño conceptual	Modelo relacional	Base de datos relacional
Entidad	Relación	Tabla o relación
Instancia	Tupla	Tupla o fla
Atributo	Atributo	Atributo o columna
Dominio	Dominio	Dominio
Identifcación inequívoca	Clave primaria	Clave primaria
Dependencia funcional	Clave foránea o ajena	Clave foránea o ajena
Valor desconocido	Valor desconocido	Valor nulo

Fuente: Reinosa, Maldonado, Muñoz, Damiano y Abrutsky, 2012, p. 35.

Tabla 2. Esta tabla presenta un cuadro de equivalencia de términos entre las distintas etapas de la definición de un modelo de datos relacional.

2. Relaciones

Una relación es una asociación entre diferentes tablas. Por ejemplo, se puede definir una relación que asocie al cliente Santos con el préstamo P-15. Esta relación especifica que Santos es un cliente con el préstamo número P-15, como vemos en la tabla 3 (Silberschatz et al., 2002).

Tabla 3: Ejemplo de relación entre la tabla clientes y la tabla *préstamos*

Id cliente	Nombre - Cliente	Dirección	Ciudad	Teléfonos
32.112.312	**Mario Santos**	**Colón 278**	**Rosario**	**4615666,4206633**
46.566.778	**Ana María Suarez Paz**	**La paz 676**	**Rosario**	**4813346**
16.083.322	**Roxana Becú Gómez**	**General Paz 44**	**Córdoba**	**4252500,4252501**

Id préstamo	Monto	Id Cliente
P-15	**10.000**	**32.112.312**
P-16	**38.000**	**32.112.312**
P-18	**7.000**	**16.083.322**

Fuente: Elaboración propia.

Tabla 3. Vemos lo descripto anteriormente de columnas y filas de la tabla *clientes*.

En el caso de la tabla 3, vemos una relación entre clientes y préstamos, en la que un cliente puede tener ningún préstamo (como el cliente Suarez Paz), un cliente tiene un préstamo solamente (como el cliente Becú Gómez) y otro cliente tiene dos préstamos (como el cliente Santos). En este caso, la relación es 1:M, es decir, un cliente puede tener

desde 0 a muchos préstamos y un préstamo pertenecer a un solo cliente. La forma de relacionar estas tablas, con la relación “toma un préstamo” se resuelve con la clave foránea, definición ya vista en lecturas anteriores, que es, ni más ni menos, la clave primaria de clientes incluida en la tabla *préstamos*.

Podemos generalizar la situación de las relaciones entre dos tablas, o binaria, con las siguientes características.

> Para un conjunto de relaciones binarias R entre los conjuntos de entidades [tablas] A y B, la correspondencia de cardinalidades debe ser una de las siguientes:
>
> - **Uno a uno**. Una entidad [fila] en A se asocia con, a lo sumo, una entidad [fila] en B, y una fila en B se asocia con, a lo sumo, una fila en A.
> - **Uno a varios.** Una fila en A se asocia con cualquier número de filas en B (ninguna o varias). Una fila en B, sin embargo, se puede asociar con, a lo sumo, una fila en A.
> - **Varios a varios.** Una fila de A se asocia con cualquier número de filas (ninguna o varias) en B, y una fila en B se asocia con cualquier número de filas (ninguna o varias) en A. (Silberschatz et al., 2002, p. 22).

Esta cardinalidad se resuelve creando una nueva tabla que contenga las claves foráneas a las otras dos tablas más algún atributo o columna que describa esta nueva tabla.

Las claves foráneas de una tabla en una “uno a varios” se colocan solo en la tabla de la parte «varios» de la relación. Por otra parte, para las tablas uno a uno, hay dos formas de resolver esta situación, unir todos los atributos en una sola tabla o, si eso es imposible o por razones de diseño es provechoso mantener las dos tablas, entonces se decide que la única clave foránea se puede ubicar en cualquiera de las tablas. El criterio aceptado dice que es más conveniente que vayan del lado de la tabla con menos filas.

3. Tareas que se deben realizar y resultados esperados

Entendiendo las características de los elementos del modelo relacional, podremos avanzar con paso firme en la definición de las tablas, columnas y relaciones que van a formar parte de las aplicaciones de *software* que, juntos con los programas, permitirán atender las operaciones de la empresa y ayudar a la gestión de las organizaciones, ya que estos elementos reflejan su realidad y permiten tener información segura y confiable.

Entonces, como profesionales de informática, nos encontramos ya ubicados en el tema de diseño de modelos de datos relacionales, con la misión de construir los objetos que la aplicación de software requiere para mostrar y guardar la información en forma eficiente y segura.

Estamos ubicados en el ciclo de vida de un *software*. La ingeniería de software ha desarrollado numerosas estrategias metodológicas para construir este sistema, compuestos de *software* (tema del que no hablaremos porque es específico de la programación procedimental) y de tablas, columnas, claves, reglas y demás objetos definibles dentro de un motor de base de datos relacional.

Pondremos el foco de esta lectura en tratar los elementos básicos del análisis y diseño de modelo de datos, deteniéndonos en revisar el contexto de estas dos fases en el proceso completo. Veremos en detalle el análisis de E-R, una técnica originada dentro del modelo relacional.

También veremos, en otra lectura, la normalización, que nos ayudará como guía para lograr un modelo de tablas, columnas, claves primarias y foráneas dispuestas de tal manera que se reduce al mínimo la redundancia de información.

Usaremos, para explicar los conceptos y el proceso de analizar y diseñar modelos de datos, herramientas informáticas que ayudan al analista a registrar, graficar y construir al fin los objetos resultantes del proceso previsto en el ciclo de vida de los sistemas de información.

Los modelos de datos relacionales son parte de un sistema de información. Antes de comprender su desarrollo, debe comprenderse el contexto que los rodea. Esta sección describe los componentes de un sistema de información y pautas metodológicas para su desarrollo.

4. Objetivos del análisis

El ciclo de vida es un marco de referencia para definir distintas fases, describe el tipo de actividad y muestra los detalles adicionales hasta que el sistema se construye. Cada fase puede describirse como sigue:

☰ Fase de investigación preliminar

Genera el enunciado del problema y un estudio de factibilidad. El estudio de factibilidad identifica los costos y los beneficios de los sistemas. Si el sistema es factible, se otorga la aprobación para comenzar su análisis.

☰ Fase de análisis de sistemas

Genera los requerimientos que describen a los procesos, datos e interacciones con el entorno. Se usan técnicas de diagramación para documentar los procesos, datos e interacciones en el entorno. Para generar los requerimientos, se estudia el sistema actual y se entrevista a los usuarios del sistema propuesto.

☰ Fase de diseño de sistemas

Genera un plan para implementar los requerimientos de forma eficiente. Se crean las especificaciones de diseño de los procesos, datos e interacciones con el entorno. Las especificaciones de diseño se enfocan en las alternativas que optimicen los recursos, dadas las restricciones que se presenten.

☰ Fase de implementación de sistemas

Genera el código ejecutable, bases de datos y documentación para el usuario. Para implementar el sistema se codifican y prueban las especificaciones del diseño. Antes que el sistema esté funcionando, se debe crear un plan de transición del sistema actual al nuevo. Para obtener total confianza y experiencia con el nuevo sistema, la empresa debe utilizar ambos sistemas en paralelo durante algún tiempo.

☰ Fase de mantenimiento

Genera las correcciones, cambios y mejoras al sistema de información. La fase de mantenimiento inicia cuando un sistema de información comienza sus operaciones. Se distingue esta fase porque incluye actividades de todas las anteriores y se considera que debe finalizar cuando el costo de mantener estas actividades es mayor que construir un nuevo sistema de información.

Este enfoque es denominado “de cascada” y en él, una fase finaliza y recién se inicia la siguiente y es criticado por diferentes motivos. Primero, con esta forma de trabajo, una aplicación totalmente operativa no se produce hasta que termina el proceso. Al tiempo que el sistema es puesto operativo, los requerimientos pueden haber cambiado, lo que lo haría obsoleto o incompleto antes de funcionar. Segundo, generalmente existe cierta prisa para comenzar la implementación para que el producto sea visible. Dada esta prisa, es posible que no se dedique el tiempo suficiente al análisis y diseño, y se produzca un resultado de baja calidad.

La meta del proceso de desarrollo de un modelo relacional de datos es generar un modelo de datos operacional para un sistema informático, se deben definir tres vistas o esquemas (externo, conceptual e interno) y poblar las bases de datos. Las primeras dos vistas se

enfocan en una implementación eficiente.

Para lograr el modelo conceptual, es necesario trabajar en la fase de análisis, porque es preciso conocer los requerimientos y se construye, normalmente, un diagrama entidad-relación o DER, por sus siglas. Los requerimientos de datos pueden tener varios formatos, como entrevistas con los usuarios, documentación de los sistemas actuales, formularios y reportes propuestos. Este modelo debe abarcar todos los requerimientos y formatos.

En esta fase y como antecesor de las futuras tablas que se deberán definir en la fase de diseño, renovamos el concepto de **entidad**, que es todo aquel objeto físico o abstracto del cual el sistema necesita almacenar alguna información. Las entidades tienen atributos, a los que podemos considerar los antecesores de las columnas de las tablas de diseño.

En la fase del diseño, entonces, estableceremos las tablas, sus atributos, las claves primarias y foráneas. Una vez establecidos estos elementos, que podemos graficar en un modelo de tablas, deberemos aplicar la normalización, que permitirá definir, con base en las formas normales, un diseño de tablas que evite la redundancia y que logre que todas las columnas de una tabla dependan funcionalmente, es decir, que el valor de cada una dependerá del valor de la clave primaria, de toda ella, si fuera compuesta, y de ninguna otra columna que no sea clave. Con este diseño normalizado, se evitarán numerosas falencias, como las anomalías de actualización, borrado e insertado. Es de destacar que es usual que el modelo normalizado tenga más tablas que el modelo inicial, pero todas ellas más sencillas y que cumplan con estas características.

La implementación del modelo es escribir las sentencias SQL del sublenguaje DDL o de definición de datos, este tema será desarrollado en el siguiente módulo de la materia.

Finalmente, la fase de mantenimiento, posterior a la implementación del *software*, que acompaña al modelo de datos para que funcione conjuntamente, en lo que se denomina genéricamente aplicación. Esta aplicación en marcha generará necesidades de ampliación y ajuste, ya propios del ciclo de vida de cualquier aplicación, que pueden percibirse como oportunidades de mejora.

En el proceso propuesto para el ciclo de vida de sistemas, se describe una fase en la que se detallan las tareas para implementar los requerimientos de forma eficiente. Esa fase se llama:

Fase de diseño

Fase de análisis de sistemas

Fase de investigación preliminar

Fase de implementación

Fase de mantenimiento

Las claves foráneas de una tabla en una “uno a varios” se colocan solo en la tabla de la parte «uno» de la relación.

Falso.

Verdadero.

Referencias

Reinosa, E., Maldonado, C., Muñoz, R., Damiano, L. y Abrutsky, M. (2012). *Bases de datos*. Buenos Aires, AR: Alfaomega.

Silberschatz, A., Korth, H. y Sudarshan, S. (2002). *Fundamentos de bases de datos*. MX: MCGrawGill.